

P1102R0

Down with ()!

Published Proposal, 20 June 2018

This version:

<http://wg21.link/P1102R0>

Authors:

[Alex Christensen](#) (Apple)

[JF Bastien](#) (Apple)

Audience:

CWG

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Source:

<https://github.com/jfbastien/papers/blob/master/source/p1102r0.bs>

Abstract

A proposal for removing unnecessary ()'s from C++ lambdas.

Table of Contents

1 Introduction and motivation

2 Impact

3 Wording

References

Informative References

§ 1. Introduction and motivation

Currently, C++ lambdas with no parameters do not require a parameter declaration clause. The specification even contains this language in `[expr.prim.lambda]` section 8.4.5 ¶4:

If a lambda-expression does not include a lambda-declarator, it is as if the lambda-declarator were `()`.

This allows us to omit the unused `()` in simple lambdas such as this:

```
std::string s1 = "abc";
auto withParen = [s1 = std::move(s1)] () {
    std::cout << s1 << '\n';
};

std::string s2 = "abc";
auto noSean = [s2 = std::move(s2)] { // Note no syntax error.
    std::cout << s2 << '\n';
};
```

These particular lambdas have ownership of the strings, so they ought to be able to mutate it, but `s1` and `s2` are const (because the `const` operator is declared `const` by default) so we need to add the `mutable` keyword:

```
std::string s1 = "abc";
auto withParen = [s1 = std::move(s1)] () mutable {
    s1 += "d";
    std::cout << s1 << '\n';
};

std::string s2 = "abc";
auto noSean = [s2 = std::move(s2)] mutable { // Currently a syntax error.
    s2 += "d";
    std::cout << s2 << '\n';
};
```

Confusingly, the current Standard requires the empty parens when using the `mutable` keyword. This rule is unintuitive, causes common syntax errors, and clutters our code. When compiling with clang, we even get a syntax error that indicates the compiler knows exactly what is going on:

```
example.cpp:11:54: error: lambda requires '()' before 'mutable'
auto noSean = [s2 = std::move(s2)] mutable { // Currently a syntax error.
                                   ^
                                   ()
1 error generated.
```

This proposal would make these parentheses unnecessary like they were before we added `mutable`. This will apply to:

- lambda template parameters
- `constexpr`
- `mutable`
- Exception specifications and `noexcept`
- attributes
- trailing return types
- `requires`

EWG discussed this change as [\[EWG135\]](#) in [Lenexa](#) and voted 15 to 1 on forwarding to core. It became [\[CWG2121\]](#), discussed in [Kona](#) and needed someone to volunteer wording.

This paper was discussed on the EWG reflector in June, Nina Ranns provided feedback, and EWG chair agreed that the paper should move to CWG directly given previous polls.

§ 2. Impact

This change will not break existing code.

§ 3. Wording

Modify Lambda expressions [**expr.prim.lambda**] as follows:

lambda-expression :

lambda-introducer lambda-declarator requires-clause_{opt} compound-statement

~~*lambda-introducer < template-parameter-list > requires-clause_{opt} compound-statement*~~

lambda-introducer < template-parameter-list > requires-clause_{opt}

lambda-declarator requires-clause_{opt} compound-statement

lambda-introducer :

[lambda-capture_{opt}]

lambda-declarator :

(parameter-declaration-clause)_{opt} decl-specifier-seq_{opt}

noexcept-specifier_{opt} attribute-specifier-seq_{opt} trailing-return-type_{opt}

Modify §4:

If a ~~*lambda-expression*~~ *lambda-declarator* does not include a ~~*lambda-declarator*~~ *(.parameter-declaration-clause.)*, it is as if the ~~*lambda-declarator*~~ *(.parameter-declaration-clause.)* were *()*. The lambda return type is *auto*, which is replaced by the type specified by the *trailing-return-type* if provided and/or deduced from *return* statements as described in 10.1.7.4.

Keep Closure types [**expr.prim.lambda.closure**] §3 as-is:

The return type and function parameters of the function call operator template are derived from the *lambda-expression*'s *trailing-return-type* and *parameter-declaration-clause* by replacing each occurrence of *auto* in the *decl-specifiers* of the *parameter-declaration-clause* with the name of the corresponding invented *template-parameter*. The *requires-clause* of the function call operator template is the *requires-clause* immediately following *< template-parameter-list >*, if any. The trailing *requires-clause* of the function call operator or operator template is the *requires-clause* following the *lambda-declarator*, if any.

Note: The first sentence can remain as-is because the modification to [**expr.prim.lambda**] §4 create an empty *parameter-declaration-clause* if none is provided. Similarly, the second and third sentences bind the *requires-clause* unambiguously.

§ References

§ Informative References

[CWG2121]

EWG. More flexible lambda syntax. 6 May 2015. drafting. URL: <https://wg21.link/cwg2121>

[EWG135]

Herb Sutter. [tiny] Mutable is part of a lambda-declarator, so when a lambda is mutable, the parentheses aren't optional. New. URL: <https://wg21.link/ewg135>